Bitácora de uso de IA

Curso: Inteligencia Artificial, UTB. **Integrantes:** Hasmet Gonzalez Perez y Dylan Andrés Ecker López. **Herramienta:** Claude (Anthropic). **Fechas:** 15, 16, 17 y 18 de octubre de 2026.

Usamos Claude durante toda la Tarea 3, y su aporte fue grande: escribió el código del agente y del simulador, los scripts de pruebas, los diagramas y buena parte del texto del documento. Nosotros definimos el agente desde la Tarea 2, escogimos entre los dos diseños, leímos lo que se generó, ejecutamos el proyecto en nuestros equipos y corregimos lo que no nos cuadraba. Abajo va cada uso, qué entregó la IA y qué hicimos nosotros con eso.

#	Qué le pedimos a la IA	Qué entregó	Qué hicimos nosotros
1	Pedimos que nos explicara la actividad y propusiera un plan por fases.	Un plan en cuatro etapas.	Lo leímos con la guía al lado y lo ajustamos a los días que teníamos.
2	Comparar ClauseCheck-Agent y FlightPredict-Agent y recomendar uno. (15/10)	Una matriz de decisión ponderada, con sensibilidad (4,25 contra 3,35 a favor de FlightPredict).	Revisamos los puntajes, sobre todo el de viabilidad, para ver que reflejaran lo que realmente podemos programar. Estuvimos de acuerdo con la elección.
3	Implementar el agente, el simulador, el log y las pruebas. (16/10)	El código del repositorio (Python y scikit-learn), los scripts de entrenamiento, demo y pruebas, y las 65 corridas.	Instalamos las dependencias, corrimos train.py y run_tests.py y comparamos las cifras con las del documento. Leímos los módulos principales (percepción, memoria, modelo, decisión y el ciclo del agente) para poder explicarlos.
4	Diagramas de arquitectura y de secuencia.	Los archivos .dot y .mmd y los PNG.	Los comparamos con el código para ver que los mensajes del diagrama coincidieran con las llamadas reales.
5	Redactar el documento de diseño ajustado.	Un PDF con la matriz, la tabla de cambios, el diseño corregido y la sección 9.	Lo leímos completo, revisamos que las cifras salieran de los CSV y escribimos nosotros las observaciones del docente en la tabla de cambios.
6	Revisar el entregable contra la rúbrica (17/10).	Una reejecución independiente (M1 74,0 %, M2 100 %, M4 100 %, M3 46,2 % con scikit-learn 1.9.1 frente a 46,0 % original) y una lista de brechas.	Revisamos la lista y entendimos por qué M3 no cumple. Decidimos mostrarlo como está en el documento y no tocar el umbral.
7	Corregir diagramas, documento y repositorio según esa revisión.	Participante «Entorno» en los diagramas, retrain() corregido, requirements.txt con versiones fijas, el script que arma el documento, el Anexo A con el log real y la guía de defensa.	Revisamos los cambios y volvimos a generar los resultados en nuestros equipos.
8	Corregir defectos encontrados en la revisión.	Logger con JSON estricto, logs regenerados con resultados idénticos, copias sin comprimir en results/logs_muestra/, tildes en las tablas y letra más grande en los diagramas.	Volvimos a correr run_tests.py y comprobamos que las cifras no cambiaron.
9	Publicar el agente en una plataforma (17/10).	Primero una app en Hugging Face Spaces; al salir el error 403 de cuota pasamos a Streamlit Community Cloud. Quedaron streamlit_app.py y core.py, que usan el mismo código del agente.	Hicimos el despliegue en Streamlit, probamos los casos de la pestaña inicial y comprobamos que M1 y M3 coinciden con el documento.

10	Actualizar el documento final con la plataforma (18/10).	La sección 9.9, los enlaces al repositorio y a la app, y los ajustes a limitaciones y tabla de cambios; el documento se regeneró con las mismas cifras.	Abrimos el PDF, comprobamos que los enlaces funcionan y terminamos de llenar los campos correspondientes.

Los resultados numéricos salen de ejecutar el código incluido sobre un entorno simulado; no se inventaron cifras. La IA no pegó los enlaces. Podemos entender, reproducir y defender el trabajo.